primitives such as mutexes and condition variables give the test writer control over which thread gets the lock when multiple threads are waiting or which thread is notified by a `notify_one()` call on a condition variable. This would allow you to set up particular scenarios and verify that your code works as expected in those scenarios.

Some of these testing facilities would have to be supplied as part of the C++ Standard Library implementation, whereas others can be built on top of the Standard Library as part of your test harness.

Having looked at various ways of executing test code, let's now look at ways of structuring the code to achieve the scheduling you want.

10.2.5. Structuring multithreaded test code

Back in section 10.2.2 I said that you need to find ways of providing suitable scheduling for the "while" part of your tests. Now it's time to look at the issues involved in that.

The basic issue is that you need to arrange for a set of threads to each be executing a chosen piece of code at a time that you specify. In the most basic case you have two threads, but this could easily be extended to more. In the first step, you need to identify the distinct parts of each test:

- The general setup code that must be executed before anything else
- The thread-specific setup code that must run on each thread
- The actual code for each thread that you desire to run concurrently
- The code to be run after the concurrent execution has finished, possibly including assertions on the state of the code

To explain further, let's consider a specific example from the test list in section 10.2.2: one thread calling `push()` on an empty queue while another thread calls `pop()`.

The *general* setup code is simple: you must create the queue. The thread executing `pop()` has no *thread-specific* setup code. The thread-specific